

```

#
=====
=====

# Python Assignment 2: Data Structures -
List, Dictionary, Set
# & Conditional Statements
#
=====
=====

#
=====
=====

# LIST
#
=====
=====

print("=" * 50)
print("LIST")
print("=" * 50)

# 1. List creation
age_list = [24, 25, 26, 27, 28]
name_list = ["Anu", "Ravi", "Meera",
             "Kiran", "Divya"]
print("age_list:", age_list)
print("name_list:", name_list)

# 2. List operations / modifications
# a. Append "Yazhini" to name_list

```

```
name_list.append("Yazhini")
print("\na. After append:", name_list)

# b. Insert 30 at index 2 in age_list
age_list.insert(2, 30)
print("b. After insert:", age_list)

# c. Remove "Yazhini" from name_list
name_list.remove("Yazhini")
print("c. After remove:", name_list)

# d. Pop the last element from age_list
popped = age_list.pop()
print("d. Popped element:", popped, "|
age_list:", age_list)

# e. Extend age_list with [29, 30, 26]
age_list.extend([29, 30, 26])
print("e. After extend:", age_list)

# f. Sort age_list in descending order
age_list.sort(reverse=True)
print("f. Sorted (descending):", age_list)

# g. Max, min and sum of ages
print("g. Max age:", max(age_list))
print("    Min age:", min(age_list))
print("    Sum of ages:", sum(age_list))

# 3. Accessing list elements
print("\na. First element of name_list:",
name_list[0])
print("b. Last element of name_list:",
name_list[-1])
```

```

print("c. Elements from index 2 to 4:",
      name_list[2:5])
print("d. name_list in reverse order:",
      name_list[::-1])

#
=====
=====
# DICTIONARY
#
=====
=====
print("\n" + "=" * 50)
print("DICTIONARY")
print("=" * 50)

# a. Create dictionary
student_marks = {"Anu": 88, "Ravi": 75,
                 "Meera": 92, "Kiran": 68, "Divya": 81}
print("a. student_marks:", student_marks)

# b. Access mark of a specific student
print("b. Mark of Meera:",
      student_marks["Meera"])

# c. Add new student "Janani" with mark 80
student_marks["Janani"] = 80
print("c. After adding Janani:",
      student_marks)

# d. Update the mark of an older student to
82
student_marks["Ravi"] = 82
print("d. After updating Ravi's mark to

```

```

82:", student_marks)

# e. keys(), values(), items()
print("e. Keys:", student_marks.keys())
print("  Values:", student_marks.values())
print("  Items:", student_marks.items())

#
=====
=====
# SETS
#
=====
=====
print("\n" + "=" * 50)
print("SETS")
print("=" * 50)

# a. Create my_set from a list with
duplicates
my_set = set(['a', 'e', 'i', 'o', 'u', 'a',
'a', 'i'])
print("a. my_set:", my_set)
print("  Explanation: A set stores only
unique elements, so the duplicate")
print("  values 'a' and 'i' are removed. A
set is also unordered, so the")
print("  elements may be printed in any
order.")

# b. Try to change my_set[4] = 's'
print("\nb. Trying my_set[4] = 's'")
try:
    my_set[4] = 's'

```

```

except TypeError as e:
    print("    Error:", e)
    print("    Explanation: Sets are
unordered and have no index positions,")
    print("    so items cannot be accessed
or changed using an index.")

# c. Create two sets
set1 = {1, 3, 5, 7, 9}
set2 = {2, 3, 5, 8, 10}
print("\nc. set1:", set1)
print("    set2:", set2)

# d. Union and intersection
print("d. Union:", set1 | set2)          #
same as set1.union(set2)
print("    Intersection:", set1 & set2)  #
same as set1.intersection(set2)

#
=====
=====
# CONDITIONAL STATEMENTS: Performance
Category Program
#
=====
=====
print("\n" + "=" * 50)
print("PERFORMANCE CATEGORY")
print("=" * 50)

score = float(input("Enter your score (0 to
10): "))

```

```
if score < 0 or score > 10:
    print("Invalid score! Please enter a
score between 0 and 10.")
elif score > 7:
    print("Above Average: Excellent work!
Keep up the great performance.")
elif score >= 4:
    print("Average: Good effort! Keep
practicing, there's room for improvement.")
else:
    print("Below Average: Need to improve
your performance, consistent practice will
lead to better results.")
```